PeerVest

P2P Lending Robo-Advisor

Using Neural Network (Probability of Default) &
Random Forest Regression (Annualized Return)

Project Report

Contents

I. Introduction

a) Background Description

Peer-to-Peer (P2P) lending platforms such as LendingClub connect individual borrowers directly with retail investors, bypassing traditional financial intermediaries. Because these platforms operate at scale, their default risk-scoring systems group loans into broad risk buckets (grades A–G), which are too coarse for informed loan-by-loan investment decisions. This creates a systematic pricing inefficiency: high-quality loans within a risk bucket are priced identically to lower-quality ones within the same bucket.

PeerVest addresses this inefficiency by applying machine learning to publicly available LendingClub historical loan data, scoring each individual loan on two dimensions: the probability it will default (risk), and its predicted annualized return (reward). A custom Risk-Adjusted Ratio (RAR) then ranks investable loans, and the final recommendations are surfaced through a deployed interactive web application.

b) Data Description

The raw dataset is assembled from 16 quarterly LendingClub CSV files spanning 2007 Q1 to 2017 Q4. After concatenation the dataset contains:

Total Samples	Raw Features	Training Set (after cleaning)	Test Set (after cleaning)
1,765,451	**150**	**880,950**	**377,551**

Each row represents a single loan. Target variables differ by model:

- **Classification model:** *loan_status* — binary label (1 = Fully Paid, 0 = Charged Off / Default)
- **Regression model:** *annualized_return* — a continuous value derived from total payment, funded amount, and loan term duration

For live loan recommendations, a separate browseNotes CSV (downloaded from a LendingClub investor account) provides loans that are still seeking funding.

c) Data Sources

Historical completed loan data: LendingClub public data download portal.
Data referenced: LendingClub loan statistics, 2007–2019. Retrieved from https://www.lendingclub.com/info/download-data.action
Live investable loans: LendingClub Investor API / browseNotes CSV (requires investor account).

d) Data Uses

The aim of this project is to build an end-to-end automated investment recommendation system. Given a user's available funds, maximum acceptable probability of default, and minimum desired annualized return, the system:

- Scores every currently listed loan on LendingClub with a probability of default (neural network) and a predicted annualized return (random forest)
- Filters loans that do not meet the user's preferences
- Ranks remaining loans by a custom Risk-Adjusted Ratio (RAR)
- Displays the top recommended portfolio in a web UI and generates a downloadable CSV of loan IDs for direct investment on LendingClub

e) Summaries

This analysis covers five major stages: data ingestion and cleaning, feature engineering (preprocessing and one-hot encoding), classification model development (Keras neural network), regression model development (Scikit-learn Random Forest), and web application deployment on AWS EC2. The final system achieved an ROC-AUC of 0.86 for default prediction and an R^2 of 0.97 for return prediction (when payment history is available). The business conclusion is that granular ML-based loan scoring significantly outperforms the platform's grade-based system for informed investment allocation.

II. Data Analysis Process

a) Importing Data and Preparing Data for Statistical Modelling

Sixteen quarterly CSV files were loaded with Pandas and concatenated into a single DataFrame (1,765,451 rows × 150 columns). An initial inspection confirmed column consistency across files and that no two samples share the same borrower ID.

Loans with statuses indicating an unfinished repayment — Current, Late (16–30 days), Late (31–120 days), In Grace Period, and loans flagged as not meeting the credit policy — were excluded. Only Fully Paid (label 1) and Charged Off / Default (label 0) loans were retained for training.

Key preprocessing steps applied at the import stage:

- *loan_status* binarised: Fully Paid → 1; Charged Off / Default → 0
- *earliest_cr_line* converted from date string to integer (days since epoch) via `pd.to_timedelta`
- *revol_util* and *int_rate* parsed from percentage strings to float64
- *emp_length* standardised: '< 1 year' → 0; '10+ years' → 10; leading digit extracted; cast to float
- Loans with missing *loan_amnt* or *inq_last_6mths* dropped; *zip_code* and *emp_title* nulls removed
- *emp_title* grouped: titles appearing fewer than 1,600 times collapsed into 'Other', leaving 50 categories

After cleaning the dataset retained 1,258,501 rows with 103 columns. A stratified 70/30 train-test split (random_state=0) produced training and test sets of 880,950 and 377,551 loans respectively.

b) Missing Value Imputation and Feature Selection

Three column groups were defined and imputed using training-set statistics only — no information from the test set was used — to prevent data leakage:

Strategy	Example Columns	Rationale
Fill with column MAX × 5	mths_since_last_delinq, mths_since_last_record, and 10 similar recency features	A missing value means the event never occurred; assign a very large recency value to reflect this
Fill with 0	tot_coll_amt, open_acc_6m, mort_acc, pub_rec_bankruptcies, tax_liens, and 28 similar count features	Missing count features represent zero occurrences
Fill with training mean	dti, annual_inc_joint, dti_joint, il_util, all_util, bc_util, pct_tl_nvr_dlq, avg_cur_bal, revol_util	Continuous ratio features where absence reflects a missing measurement, not a zero value

String-encoded floats in twelve columns (e.g. *bc_open_to_buy*, *percent_bc_gt_75*) were explicitly cast to float64 before imputation. After imputation, no missing values remained in either set.

Feature selection removed columns providing redundant or outcome-leaking information: *term*, *verification_status*, *grade*, *emp_title* (replaced by emp_title_2), *addr_state* (replaced by

zip_code OHE), *debt_settlement_flag*, *issue_d*, and *last_pymnt_d*. The final input dimension before one-hot encoding was 88 numeric features.

c) Feature Engineering — One-Hot Encoding

Six categorical columns were one-hot encoded using Scikit-learn's `OneHotEncoder`. Encoders were fitted on X_train only and applied to both train and test sets to prevent leakage. The `handle_unknown='ignore'` parameter ensures unseen categories in live data are represented as all-zero vectors.

Column	OHE Dimensions	Notes
home_ownership	6	MORTGAGE, RENT, OWN, ANY, OTHER, NONE
purpose	14	14 loan purpose categories (debt consolidation dominant at 59%)
zip_code	930	Preferred over addr_state (51) for finer geographic signal
application_type	2	Individual / Joint App
sub_grade	35	A1–G5; more granular than grade (7 levels)
emp_title_2	50	Top 49 employer titles + 'Other' bucket
Total features after OHE	**1,132**	**Final input dimensionality for both models**

d) Model 1 — Neural Network Classification (Probability of Default)

The classification objective is to predict, for each loan, the probability that it will be fully repaid (class 1) or default (class 0). The continuous probability output — rather than a binary prediction — drives the downstream Risk-Adjusted Ratio ranking.

Logistic Regression was tried first. While classification metrics were acceptable, the `.predict_proba()` output was poorly calibrated and the model underperformed with 1,132 features and suspected non-linear interactions. PCA dimensionality reduction offered no compression benefit — explained variance dropped proportionally with components removed. A Keras neural network was selected for its ability to model arbitrary non-linear relationships and scale with high-dimensional input.

Eight architectures were iterated (v1–v8), varying layer depth, width, regularisation type, activation functions, class weight balancing, and batch size. The best-performing model (v8) uses the following architecture:

Layer	Units	Activation	Regularisation
Input	1,107	Sigmoid	—
Dense 1	1,000	Sigmoid	L2 kernel
Dense 2	300	Sigmoid	—
Dense 3	50	Sigmoid	L2 kernel
Output	1	Sigmoid	L2 kernel

Training parameters: Loss = Binary Cross-Entropy; Optimizer = Adam; Early Stopping (monitor: validation loss); Class Weight Balancing to address the ~80% Fully Paid / ~20% Default class imbalance; Batch Size = 5,000. Training was accelerated using a Google Cloud NVIDIA Tesla GPU.

Underperformers: 3 Scikit-Learn Logistic Regression variants, 7 earlier Keras architectures (v1–v7).

e) Model 2 — Random Forest Regression (Annualized Return)

The target variable for regression is a custom Annualized Return metric computed from completed loan data:

> $$AR = (Total\ Payment / Funded\ Amount) ^ (365 / D) - 1$$

where *D* is the number of days between loan issue date and date of last payment plus 30. This annualises total return across varying loan terms into a comparable annual percentage.

Linear Regression was fitted first but overfit — strong training performance degraded on out-of-sample future data. Ridge Regression (L2 regularisation) corrected overfitting but metrics remained modest. A Random Forest was selected for its ability to bootstrap feature subsets, aggregate many weak trees, and handle high-dimensional tabular data without extensive manual feature engineering.

Five Random Forest configurations (v1–v5) were evaluated. The best model (v5) was tuned via GridSearchCV with the following optimal hyperparameters:

n_estimators	max_depth	min_samples_leaf	min_samples_split
100	**15**	**4**	**2**

Underperformers: 1 Linear Regression, 3 Ridge Regression variants, 1 Keras neural network, and 4 earlier Random Forest configurations (v1–v4).

For live loan recommendations (browseNotes data), total payment history is unavailable by definition. The model therefore uses a reduced feature set excluding payment-related columns, achieving lower but still operationally useful accuracy.

f) Model Evaluation

Model 1 — Classification (Probability of Default):

Metric	Class: Fully Paid	Overall
Precision	0.943	—
Recall	0.979	—
F1-Score	0.961	—
ROC-AUC	—	0.86
Probability Calibration	—	Well-calibrated

Model 2 — Regression (Annualized Return):

Metric	Without Payment History	With Payment History
R-Squared	0.56	0.97
MSE	0.02	0.001
RMSE	0.16	0.04

The without-payment-history configuration is the operationally relevant one for live recommendations, since new listings have no repayment history. The with-payment-history result validates the model's ceiling and confirms that the feature set is highly informative when full data is available.

g) Risk-Adjusted Ratio (RAR) and Deployment

To rank filtered loans for portfolio recommendation, a custom risk-adjusted metric is defined:

> $$RAR = (\text{Portfolio Expected Return} - \text{Risk-Free Rate}) / \text{Portfolio Weighted Average Probability of Default}$$

This is analogous to the Sharpe Ratio but substitutes the Probability of Default (from the neural network) as the denominator risk measure rather than historical return volatility. The hypothesis is that the model's default probability is a more precise and forward-looking risk signal than backward-looking standard deviation.

The complete production pipeline:

- User inputs: available funds ($), maximum probability of default (%), minimum desired annualized return (%)
- Live loan data is ingested, cleaned, OHE-encoded, and fed through both trained models
- Loans not meeting the user's thresholds are filtered out
- Remaining loans are ranked by RAR, descending
- Top loans are displayed in the web UI; the full filtered portfolio is downloadable as a CSV of loan IDs for direct investment on LendingClub

Deployment stack: Flask web application hosted on an Amazon Web Services EC2 instance. Frontend built with HTML, CSS, and Brython (Python in the browser). Both trained models are serialised with joblib and loaded at application startup.

III. Conclusion

PeerVest demonstrates that publicly available P2P lending data, combined with a carefully designed ML pipeline, can produce loan-level risk and return predictions that meaningfully outperform the platform's own grade-based bucketing system. The dual-model architecture — a neural network for risk, a random forest for return — allows the system to jointly optimise on both dimensions, which single-model approaches cannot achieve.

Key findings:

- A 4-layer Keras neural network achieves ROC-AUC 0.86 for default probability prediction with well-calibrated probability outputs that drive reliable portfolio ranking

- A Random Forest with GridSearch-tuned hyperparameters achieves $R^2 = 0.97$ for annualized return prediction when payment history is available, and a still-useful $R^2 = 0.56$ for live unlisted loans
- The Risk-Adjusted Ratio (RAR) provides a principled, model-driven framework for comparing loans on a risk-adjusted return basis, analogous to the Sharpe Ratio in traditional portfolio management
- The end-to-end system — from raw CSV ingestion to deployed web application — validates the complete data engineering and ML lifecycle, including pipeline reproducibility, feature engineering at scale, model serialisation, and API-based serving

Future directions include incorporating real-time LendingClub Investor API data feeds, extending to the Prosper Marketplace, and exploring survival analysis to better model time-to-default dynamics.

IV. References

LendingClub Corporation. (2019). Loan statistics data download, 2007–2019. Retrieved from https://www.lendingclub.com/info/download-data.action

Chollet, F. et al. (2015). Keras: Deep Learning for Humans. https://keras.io

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.